同芯同行

Chip in, big win

UVM Scoreboard 常考题 / 真题汇总

Part 3 / 3 · 题目 18 ~ 25

SystemVerilog · UVM 1.2 / IEEE 1800.2

本册涵盖题目 18-25

Bypass动态切换（18）/ USB背靠背时序（19）/ ADC模糊评分（20）

ARQ重传状态机（21）/ DMA多Agent（22）/ 防火墙黑名单（23）

高速压缩缓存（24）/ 三层继承体系（25）

同芯同行，真题先行

题目 18 带 Scoreboard Bypass 的直通模式验证

难度：中级

标签：bypass / 运行时动态配置 / uvm_config_db / 双端口

【题目描述】

验证一个支持两种工作模式的数据处理模块：处理模式下对输入数据执行特定的变换操作（例如字节翻转），Bypass 模式下数据直接透传不做任何修改。两种模式在测试过程中可以动态切换，切换后立即对后续事务生效。Scoreboard 需要同时监控输入端和输出端，根据当前生效的模式选择对应的比对策略，同时提供一个公共方法供 Virtual Sequence 在运行时切换模式，以便验证模式切换时的 DUT 行为是否正确。本题考察 Scoreboard 的双端口监控设计和运行时动态配置能力。

【考察重点与实现要点】

> 动态配置的关键：write() 中每次都读取 bypass_mode 当前值，不能在 build_phase 固化
>
> set_bypass() 是公共接口，允许 virtual sequence 在测试中途动态切换模式验证切换行为
>
> input_q 记录输入事务，write_out() 中 pop 取出与输出配对，保证一对一比对
>
> bypass 模式下 out.data !== in.data 说明 DUT 在不应处理时进行了数据修改
>
> 模式切换时打印 uvm_info，便于调试时确认切换时机

【完整答案代码】

```
// 方法论：input（数据处理模块输入端）→ analysis_fifo 缓存输入事务
// output（数据处理模块输出端）→ imp，write() 立即从 fifo 取输入事务，按当前模式比对
class bypass_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(bypass_scoreboard)

  // 输入侧：analysis_fifo 缓存输入事务
  uvm_analysis_export #(data_item) input_export;
  uvm_tlm_analysis_fifo #(data_item) input_fifo;

  // 输出侧：imp，write() 立即处理
  uvm_analysis_imp #(data_item, bypass_scoreboard) output_imp;

  bit bypass_mode = 0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
    void'(uvm_config_db#(bit)::get(this, "", "bypass_mode", bypass_mode));
    `uvm_info("SB", $sformatf("初始模式: %s", bypass_mode ? "BYPASS" : "PROCESS"), UVM_NONE)
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction

  // 考点：公共接口，允许 virtual sequence 运行时切换模式
  function void set_bypass(bit en);
    bypass_mode = en;
    `uvm_info("SB", $sformatf("模式切换 -> %s", en ? "BYPASS" : "PROCESS"), UVM_LOW)
  endfunction

  // output 端：write() 中从 fifo 取输入事务，按当前 bypass_mode 选择比对路径
  function void write(data_item actual);
```

```
    data_item inp;
    if (!input_fifo.try_get(inp)) begin
      `uvm_error("SB", "收到输出但输入 fifo 为空！") return;
    end

    if (bypass_mode) begin
      // Bypass 模式：输出必须与输入完全一致
      if (actual.data !== inp.data)
        `uvm_error("SB", $sformatf(
          "Bypass 模式数据被修改！in=0x%0h out=0x%0h",
          inp.data, actual.data))
    end else begin
      // 处理模式：输出必须与参考模型计算结果一致
      data_item ref_out = apply_ref_model(inp);
      if (actual.data !== ref_out.data)
        `uvm_error("SB", "处理结果与参考模型不匹配")
    end
  endfunction

  function data_item apply_ref_model(data_item in);
    data_item out = data_item::type_id::create("ref");
    out.data = new[in.data.size()];
    foreach (in.data[i])
      out.data[in.data.size()-1-i] = in.data[i]; // 字节翻转
    return out;
  endfunction

  function void check_phase(uvm_phase phase);
    if (input_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "输入 fifo 有 %0d 个事务未被消费！", input_fifo.size()))
  endfunction
endclass
```

【关键提示】

input 侧用 analysis_fifo；output 侧用 imp。write() 中 try_get 取输入事务，根据当前 bypass_mode 选择比对路径。set_bypass() 是公共接口，允许 virtual sequence 在运行时切换模式并验证切换行为。

题目 19 背靠背事务窗口检测 Scoreboard

难度：中级

标签：时序验证 / $realtime / SOF计数 / USB / CRC16

【题目描述】

验证 USB Full-Speed 控制器的帧间距时序要求。USB 协议规定同一端点（endpoint）的两次数据传输之间必须有至少 2 个 SOF（Start Of Frame）包的间隔，对应约 2ms 的物理时间。Scoreboard 需要按端点独立追踪每次传输时的 SOF 计数和绝对时间戳，在新传输到来时检查两种时间维度的间隔是否均满足要求。同时需要对每个数据包的 CRC16-IBM 字段进行校验。本题考察两种时间度量（SOF 计数与 $realtime）在时序验证中的配合使用，以及按端点独立追踪状态的数据结构设计。

【考察重点与实现要点】

> $realtime 返回 real 型时间（含小数点），单位与时间刻度一致，适合精确时间差计算
> SOF 计数是协议层度量，$realtime 是物理层度量，两者配合更严谨
> CRC16-IBM：多项式 0xA001（0x8005 的位反转），初值 0xFFFF，逐 bit 移位计算
> 关联数组 last_xfer_time[ep] 和 last_sof_at_xfer[ep] 按端点号分别记录，端点间互不影响
> write() 是 function，$realtime 调用无副作用，可以在 function 内安全使用

【完整答案代码】

```
class usb_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(usb_scoreboard)

  uvm_analysis_imp #(usb_item, usb_scoreboard) analysis_export;

  realtime last_xfer_time [int]; // 按端点号索引
  int last_sof_at_xfer [int];
  int sof_count = 0;
  int MIN_SOF_GAP = 2;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    void'(uvm_config_db#(int)::get(this, "", "min_sof_gap", MIN_SOF_GAP));
  endfunction

  function void write(usb_item pkt);
    // SOF 仅计数，不做任何比对
    if (pkt.pid == SOF) begin
      sof_count++; return;
    end

    if (pkt.pid == DATA0 || pkt.pid == DATA1) begin
      int ep = pkt.endpoint;
      // 考点：$realtime 获取当前仿真时间（real型）
      realtime now = $realtime;

      if (last_xfer_time.exists(ep)) begin
        int sof_gap = sof_count - last_sof_at_xfer[ep];
        realtime time_gap = now - last_xfer_time[ep];

        // 协议层：SOF 间隔检查
        if (sof_gap < MIN_SOF_GAP)
          `uvm_error("SB", $sformatf(
            "EP%0d 背靠背违规: 间隔 %0d 个 SOF < 要求 %0d 个",
```

```
            ep, sof_gap, MIN_SOF_GAP))

        // 物理层：时间间隔冗余校验
        if (time_gap < 1.9ms)
          `uvm_warning("SB", $sformatf(
            "EP%0d 时间间隔 %.3f ms < 2 ms", ep, time_gap/1ms))
      end

      last_xfer_time[ep] = now;
      last_sof_at_xfer[ep] = sof_count;

      // 考点：CRC16-IBM，多项式 0xA001
      if (!check_crc16(pkt.data, pkt.crc16))
        `uvm_error("SB", $sformatf("EP%0d USB 数据 CRC16 校验失败", ep))
    end
  endfunction

  function bit check_crc16(byte unsigned data[], bit [15:0] crc);
    bit [15:0] calc = 16'hFFFF;
    foreach (data[i]) begin
      calc ^= {8'h0, data[i]};
      repeat (8)
        if (calc[0]) calc = (calc >> 1) ^ 16'hA001;
        else calc >>= 1;
    end
    return (calc === crc);
  endfunction
endclass
```

【关键提示】

$realtime 返回 real 型时间，可在 function 中安全调用。SOF 计数（协议层）和 $realtime（物理层）双重校验更严谨。CRC16-IBM 的多项式是 0xA001（0x8005 的位反转），初值 0xFFFF，注意不要与 CRC16-CCITT（0x1021）混淆。

题目 20 基于评分机制的模糊比对 Scoreboard

难度：高级

标签：模糊比对 / ADC / 分级评分 / LSB误差 / config_db

【题目描述】

验证一个 12 位模数转换器（ADC）控制器。由于 ADC 存在固有量化误差和温漂特性，输出码字允许与理论值存在一定偏差，不能采用精确匹配的方式进行验证。Scoreboard 实现一套误差分级评分机制：偏差为 0 得满分 100 分，偏差在警告阈值以内（可配置）降级并得分递减，偏差超过失败阈值则记为失败得 0 分，最终统计所有样本的综合质量评分。误差阈值通过 uvm_config_db 配置，使同一 Scoreboard 可适应不同精度等级的 ADC。本题考察模糊比对（fuzzy comparison）和分级评分系统的设计实现。

【考察重点与实现要点】

> 期望码字必须用浮点计算：int'($rtoi(vin_mv * 4096.0 / 3300.0)) 避免整数截断引入系统误差
> $abs(error_lsb) 对正负偏差一视同仁，符合 ADC 规格中"对称误差"的定义
> score_sum / total_cnt 是综合质量分，比单纯 pass/fail 更能反映 ADC 整体性能水平
> warn_lsb 和 fail_lsb 从 config_db 读取，使同一 scoreboard 适应不同精度等级的 ADC
> total_cnt 归零保护：计算通过率时用 (total_cnt > 0) ? ... : 0.0

【完整答案代码】

```
class adc_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(adc_scoreboard)

  uvm_analysis_imp #(adc_item, adc_scoreboard) analysis_export;

  int total_cnt = 0, exact_cnt = 0, warn_cnt = 0, fail_cnt = 0;
  real score_sum = 0.0;
  int WARN_LSB = 2; // 警告阈值（<=）
  int FAIL_LSB = 4; // 失败阈值（>）

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    void'(uvm_config_db#(int)::get(this, "", "warn_lsb", WARN_LSB));
    void'(uvm_config_db#(int)::get(this, "", "fail_lsb", FAIL_LSB));
    `uvm_info("SB", $sformatf(
      "ADC 容差: 警告<=%0d LSB 失败>%0d LSB", WARN_LSB, FAIL_LSB), UVM_NONE)
  endfunction

  function void write(adc_item pkt);
    int unsigned exp_code;
    int error_lsb;
    real score;

    // 考点：浮点计算，避免整数截断误差
    exp_code = int'($rtoi(pkt.vin_mv * 4096.0 / 3300.0));
    error_lsb = int'(pkt.dout) - int'(exp_code);
    total_cnt++;

    if ($abs(error_lsb) == 0) begin
      exact_cnt++; score = 100.0;
    end else if ($abs(error_lsb) <= WARN_LSB) begin
      warn_cnt++;
      score = 100.0 - 10.0 * real'($abs(error_lsb));
      `uvm_info("SB", $sformatf(
```

```
        "ADC 偏差 %+0d LSB vin=%.1f mV exp=%0d act=%0d",
        error_lsb, pkt.vin_mv, exp_code, pkt.dout), UVM_MEDIUM)
    end else begin
      fail_cnt++; score = 0.0;
      `uvm_error("SB", $sformatf(
        "ADC 误差超限 %+0d LSB > 阈值 %0d vin=%.1f mV",
        error_lsb, FAIL_LSB, pkt.vin_mv))
    end
    score_sum += score;
  endfunction

  function void report_phase(uvm_phase phase);
    real avg = (total_cnt > 0) ? score_sum / total_cnt : 0.0;
    `uvm_info("SB", $sformatf(
      "
=== ADC 质量报告（共 %0d 个样本）===
"
      " 精确匹配: %0d (%.1f%%)
"
      " 微小偏差: %0d (%.1f%%)
"
      " 超标失败: %0d (%.1f%%)
"
      " 综合质量分: %.2f / 100.0",
      total_cnt,
      exact_cnt, 100.0*exact_cnt/(total_cnt+1),
      warn_cnt, 100.0*warn_cnt/(total_cnt+1),
      fail_cnt, 100.0*fail_cnt/(total_cnt+1), avg), UVM_NONE)
  endfunction
endclass
```

【关键提示】

模拟量验证不能用精确比对，要定义合理容差。期望码字用浮点计算避免系统误差。质量评分系统比简单 pass/fail 提供更丰富的诊断信息：综合分接近100说明 ADC
整体性能良好，即使偶有小偏差；综合分骤降说明存在系统性误差需要定位。

题目 21 带重试机制的重传验证 Scoreboard

难度：高级

标签：ARQ重传 / 状态机 / FIRST_TX / RETRANSMIT / check_phase

【题目描述】

验证带有 ARQ（Automatic Repeat reQuest，自动重传请求）机制的通信控制器。发送方在超时未收到 ACK 或收到 NACK 时会重传数据包；每次重传的内容必须与首次发送完全一致。Scoreboard 需要追踪每个数据包从首次发送、经历若干次重传，到最终收到 ACK 确认的完整生命周期，包括：验证重传数据与首次发送数据的一致性，统计各包的重传次数是否超过最大允许阈值，以及在仿真结束时验证所有发出的包均已收到确认。本题考察跨多个事务的状态机追踪与生命周期管理。

【考察重点与实现要点】

> 用 type_e 枚举区分事务类型比用 is_retx 标志位更清晰、更可扩展
>
> RETRANSMIT 必须先 exists() 检查，防止无首次记录时访问关联数组
>
> check_phase 检查 first_tx.size() == 0，保证所有发出的包都收到了 ACK
>
> retry_cnt > MAX_RETRY 报错后仍继续跟踪，不 return，让后续 ACK 能正常清理 entry
>
> 重传数据一致性验证（payload 比对）是 ARQ 验证的核心：确保重传内容未被篡改

【完整答案代码】

```
class arq_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(arq_scoreboard)

  uvm_analysis_imp #(arq_item, arq_scoreboard) analysis_export;

  arq_item first_tx [int]; // id -> 首次发送事务
  int retry_cnt [int]; // id -> 重传次数
  int MAX_RETRY = 3;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    void'(uvm_config_db#(int)::get(this, "", "max_retry", MAX_RETRY));
  endfunction

  // 考点：按事务类型的状态机分支
  function void write(arq_item pkt);
    case (pkt.type_e)
      FIRST_TX: begin
        if (first_tx.exists(pkt.id))
          `uvm_warning("SB", $sformatf("ID=%0d 重复首次发送", pkt.id))
        first_tx[pkt.id] = pkt;
        retry_cnt[pkt.id] = 0;
        `uvm_info("SB", $sformatf("首次发送 ID=%0d", pkt.id), UVM_HIGH)
      end

      RETRANSMIT: begin
        // 考点：先 exists() 检查，防止空指针
        if (!first_tx.exists(pkt.id)) begin
          `uvm_error("SB", $sformatf(
            "重传 ID=%0d 无首次发送记录！", pkt.id)) return;
        end
        if (pkt.payload !== first_tx[pkt.id].payload)
          `uvm_error("SB", $sformatf(
            "重传数据与首次不一致！ID=%0d", pkt.id))
```

```
        retry_cnt[pkt.id]++;
        if (retry_cnt[pkt.id] > MAX_RETRY)
          `uvm_error("SB", $sformatf(
            "ID=%0d 重传次数 %0d 超过阈值 %0d",
            pkt.id, retry_cnt[pkt.id], MAX_RETRY))
        else
          `uvm_info("SB", $sformatf(
            "第 %0d 次重传 ID=%0d", retry_cnt[pkt.id], pkt.id), UVM_LOW)
      end

      ACK_RX: begin
        if (first_tx.exists(pkt.id)) begin
          `uvm_info("SB", $sformatf(
            "ID=%0d 确认成功，共重传 %0d 次",
            pkt.id, retry_cnt[pkt.id]), UVM_LOW)
          first_tx.delete(pkt.id);
          retry_cnt.delete(pkt.id);
        end
      end

      NACK_RX:
        `uvm_info("SB", $sformatf(
          "收到 NACK ID=%0d，等待重传", pkt.id), UVM_LOW)
    endcase
  endfunction

  // 考点：check_phase 验证所有包都已收到 ACK
  function void check_phase(uvm_phase phase);
    if (first_tx.size() != 0)
      `uvm_error("SB", $sformatf(
        "%0d 个数据包已发出但未收到 ACK！", first_tx.size()))
    else
      `uvm_info("SB", "所有数据包均已收到 ACK", UVM_NONE)
  endfunction
endclass
```

【关键提示】

ARQ scoreboard 的核心是跨事务状态追踪：关联数组 first_tx[id] 在 FIRST_TX 时建立，在 ACK_RX 时销毁，check_phase 验证全部清空。重传数据一致性验证（payload 比对）防止重传内容被意外修改。超阈值报错后不 return，让 ACK 能正常清理，防止 check_phase 误报。

题目 22 虚拟 Scoreboard（跨 Agent 数据关联）

难度：高级

标签：DMA / 四路imp_decl / 多Agent / 中断触发比对

【题目描述】

验证一个 DMA 子系统的端到端数据搬运功能。完整流程如下：CPU 通过 AHB 总线向 DMA 控制器写入配置寄存器（源地址、目标地址、传输长度及启动位）；DMA 启动后从源内存读取数据并写入目标内存；传输完成后 DMA 拉高中断线通知 CPU。Scoreboard 需要同时监控 AHB 配置总线、源内存总线、目标内存总线、中断线共四路观测点，将跨越四条总线的事务关联起来，在中断触发时完成数据的端到端比对，验证搬运数据的完整性和正确性。本题考察多 Agent 场景下跨总线事务的关联与状态机管理。

【考察重点与实现要点】

> 方法论：src（DMA 读取源内存 = 输入）→ analysis_fifo；dst（DMA 写入目标内存 = 输出）→ imp
> write_dst() 中 try_get 从 src_fifo 取源端字节，立即与写入字节逐一比对
> ahb_imp 仅解析配置命令，管理通道状态机（IDLE/CONFIGURED/RUNNING/DONE）
> intr_imp 验证 src_fifo 已全部消费（说明所有源字节都已找到对应目标字节）
> check_phase 检查 src_fifo.size()==0 且无通道处于 RUNNING 状态

【完整答案代码】

```
// 方法论：ahb（配置总线）→ imp，解析启动命令更新状态
// src（源内存，DMA 读取端 = 输入）→ analysis_fifo，按通道各一个 queue 缓存期望数据
// dst（目标内存，DMA 写入端 = 输出）→ imp，write() 立即从 src_fifo 取对应字节比对
// intr（中断）→ imp，触发时验证所有通道数据已完整消费
`uvm_analysis_imp_decl(_ahb)
`uvm_analysis_imp_decl(_dst)
`uvm_analysis_imp_decl(_intr)

class dma_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(dma_scoreboard)

  // AHB 配置总线：imp（控制流，不参与数据比对）
  uvm_analysis_imp_ahb #(ahb_item, dma_scoreboard) ahb_imp;

  // 源内存（DMA 读取 = 输入侧）：analysis_fifo，按通道编号各一个 queue
  uvm_analysis_export #(mem_item) src_export;
  uvm_tlm_analysis_fifo #(mem_item) src_fifo; // 所有通道共用，按 addr 区分

  // 目标内存（DMA 写入 = 输出侧）：imp，write() 立即比对
  uvm_analysis_imp_dst #(mem_item, dma_scoreboard) dst_imp;

  // 中断线：imp，触发时做完整性检查
  uvm_analysis_imp_intr #(intr_item, dma_scoreboard) intr_imp;

  typedef enum {IDLE, CONFIGURED, RUNNING, DONE} dma_state_e;
  typedef struct {
    dma_state_e state;
    byte src_q[$]; // 该通道已读取的源数据（按序存储）
  } dma_ch_t;

  dma_ch_t tasks[int];
  int cur_ch = 0;
  int match_bytes, err_bytes;

  function void build_phase(uvm_phase phase);
```

```
  super.build_phase(phase);
  ahb_imp = new("ahb_imp", this);
  src_export = new("src_export", this);
  src_fifo = new("src_fifo", this);
  dst_imp = new("dst_imp", this);
  intr_imp = new("intr_imp", this);
endfunction

function void connect_phase(uvm_phase phase);
  src_export.connect(src_fifo.analysis_export);
endfunction

// AHB：解析配置寄存器写入，管理通道状态
function void write_ahb(ahb_item pkt);
  if (!pkt.hwrite) return;
  case (pkt.haddr)
    DMA_CH_REG: cur_ch = pkt.hwdata;
    DMA_SRC_REG: tasks[cur_ch].state = CONFIGURED;
    DMA_START_REG: if (pkt.hwdata[0]) begin
      tasks[cur_ch].state = RUNNING;
      tasks[cur_ch].src_q.delete();
      `uvm_info("SB", $sformatf("CH%0d DMA 启动", cur_ch), UVM_LOW)
    end
  endcase
endfunction

// 目标内存 output 端：write() 立即从 src_fifo 取期望字节比对
function void write_dst(mem_item pkt);
  mem_item src_pkt;
  if (!pkt.wr_en) return; // 只关心 DMA 写操作

  if (!src_fifo.try_get(src_pkt)) begin
    `uvm_error("SB", "DMA 写目标内存，但源 fifo 为空！")
    return;
  end
```

```
    if (pkt.data !== src_pkt.data) begin
      err_bytes++;
      `uvm_error("SB", $sformatf(
        "DMA 数据错误: src=0x%0h dst=0x%0h",
        src_pkt.data, pkt.data))
    end else
      match_bytes++;
  endfunction

  // 中断：验证源 fifo 已被全部消费（说明所有数据都已比对）
  function void write_intr(intr_item pkt);
    if (!pkt.intr_line) return;
    if (src_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "CH%0d 中断到达但源 fifo 仍有 %0d 字节未消费！",
        cur_ch, src_fifo.size()))
    else begin
      tasks[cur_ch].state = DONE;
      `uvm_info("SB", $sformatf(
        "CH%0d DMA 完成，比对 %0d 字节，错误 %0d 字节",
        cur_ch, match_bytes, err_bytes), UVM_NONE)
    end
  endfunction

  function void check_phase(uvm_phase phase);
    if (src_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "源 fifo 有 %0d 字节未被目标侧消费！", src_fifo.size()))
    foreach (tasks[ch])
      if (tasks[ch].state == RUNNING)
        `uvm_error("SB", $sformatf(
          "CH%0d DMA 已启动但未收到中断！", ch))
  endfunction
endclass
```

【关键提示】

src 是 DMA 的数据输入端（DMA 从源内存读取）→ analysis_fifo；dst 是 DMA 的数据输出端（DMA 向目标内存写入）→ imp。write_dst() 中 try_get 取源端字节，立即与写入字节比对。中断到达时验证 src_fifo 已全部消费。

题目 23 带黑名单过滤的 Scoreboard

难度：初级

标签：防火墙 / 黑名单 / CIDR子网掩码 / 双向验证

【题目描述】

验证网络防火墙芯片的 IP 地址过滤功能。防火墙维护一张 IP 黑名单规则表，规则以 CIDR 格式表示（例如 192.168.0.0/24 代表该子网段内的所有地址）。对于每个到达的数据包，DUT 应当丢弃来自黑名单地址的包，放行合法地址的包且不得修改数据内容。Scoreboard 独立维护规则表，对每个包计算期望的处理结果，并与 DUT 实际行为进行比对。本题考察 CIDR 子网掩码的计算实现，以及防火墙验证中必须同时覆盖的两类错误模式：漏放（black-listed packet passed through）和误拦（legitimate packet blocked）。

【考察重点与实现要点】

> CIDR 掩码计算：shift = 32 - prefix_len；mask = ~((32'h1 << shift) - 1)，prefix_len=32时需特判
> 双向验证：false_pass（漏放，黑名单未拦）和 false_block（误拦，合法包被拦）各自独立统计
> 放行包还需验证 out_data === in_data，防止数据在放行过程中被篡改
> add_rule() 提供动态添加规则接口，允许测试过程中修改防火墙规则验证动态更新行为
> should_block 基于黑名单计算（激励意图），dut_dropped 是 DUT 实际行为，两者不能混淆

【完整答案代码】

```
class firewall_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(firewall_scoreboard)

  uvm_analysis_imp #(pkt_item, firewall_scoreboard) analysis_export;

  typedef struct packed {
    bit [31:0] ip;
    bit [5:0] prefix_len;
  } rule_t;
  rule_t blacklist[$];

  int blocked_cnt = 0, passed_cnt = 0;
  int false_pass = 0, false_block = 0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
    add_rule(32'hC0A80000, 24); // 192.168.0.0/24
    add_rule(32'h0A000000, 8); // 10.0.0.0/8
    add_rule(32'hAC100000, 12); // 172.16.0.0/12
  endfunction

  function void add_rule(bit [31:0] ip, bit [5:0] pfx);
    rule_t r; r.ip = ip; r.prefix_len = pfx;
    blacklist.push_back(r);
  endfunction

  // 考点：CIDR 子网掩码匹配
  function bit is_blacklisted(bit [31:0] src_ip);
    foreach (blacklist[i]) begin
      bit [31:0] mask;
      int shift = 32 - int'(blacklist[i].prefix_len);
      // 考点：prefix_len=32 时 shift=0，无需特判
      mask = (shift >= 32) ? 32'h0 : ~((32'h1 << shift) - 1);
      if ((src_ip & mask) == (blacklist[i].ip & mask)) return 1;
```

```
    end
    return 0;
  endfunction

  function void write(pkt_item pkt);
    bit should_block = is_blacklisted(pkt.src_ip);

    if (should_block) begin
      if (!pkt.dut_dropped) begin
        false_pass++;
        `uvm_error("SB", $sformatf(
          "黑名单 IP 未被拦截！src=0x%0h", pkt.src_ip))
      end else
        blocked_cnt++;
    end else begin
      if (pkt.dut_dropped) begin
        false_block++;
        `uvm_error("SB", "合法 IP 被错误拦截！")
      end else begin
        passed_cnt++;
        if (pkt.out_data !== pkt.in_data)
          `uvm_error("SB", "放行包数据被篡改！")
      end
    end
  endfunction

  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "防火墙验证: 正确拦截=%0d 正确放行=%0d 漏放=%0d 误拦=%0d",
      blocked_cnt, passed_cnt, false_pass, false_block), UVM_NONE)
  endfunction
endclass
```

【关键提示】

防火墙验证的核心是双向覆盖：漏放（false_pass，黑名单包未被拦截）和误拦（false_block，合法包被错误拦截）是两类完全不同的故障模式，必须分别统计。CIDR 掩码计算：shift=32-prefix_len，mask=~((1<<shift)-1)，当 prefix_len=0（匹配所有IP）时 shift=32，需注意 SV 中移位32位的行为。

题目 24 带预测缓存的高速 Scoreboard（性能优化）

难度：高级

标签：性能优化 / 哈希缓存 / 批量处理 / semaphore / 仿真加速

【题目描述】

验证一个高带宽数据压缩引擎，每个时钟周期输出一个 256 位数据块，吞吐量极高。Scoreboard 需要对每个输出块调用参考模型计算期望的压缩结果并比对，但参考模型的计算开销较大，若在每次 write() 调用中直接执行，将严重拖慢仿真速度。本题要求通过三项优化策略构建高性能 Scoreboard：将 write() 设计为只做入队的轻量操作，在 run_phase 中积累足够数量后批量处理，并使用哈希缓存对相同输入只计算一次参考结果并复用。本题考察高吞吐量验证场景下 Scoreboard 的性能优化架构。

【考察重点与实现要点】

> write() 是 function，保持轻量（只入队），run_phase 是 task，做重计算
>
> semaphore 保护共享队列：write() 和 run_phase 真正并发，不加保护会数据竞争
>
> $sformatf("%0h", data) 将输入数据转为十六进制字符串作为哈希 key
>
> 批量等待 wait(size>=16) 减少 run_phase 的 wait 唤醒次数，提升吞吐
>
> 缓存命中率是性能指标：命中率高说明激励有大量重复输入，缓存效果显著

【完整答案代码】

```
// 方法论：input（压缩引擎输入）→ analysis_fifo 缓存原始输入数据
// output（压缩引擎输出）→ imp，write() 从 fifo 取原始数据，查缓存或计算期望，立即比对
// 哈希缓存避免对相同输入重复调用参考模型，大幅提升仿真性能
class compress_scoreboard extends uvm_scoreboard;
  `uvm_component_utils(compress_scoreboard)

  // 输入侧：analysis_fifo 缓存原始输入数据
  uvm_analysis_export #(compress_item) input_export;
  uvm_tlm_analysis_fifo #(compress_item) input_fifo;

  // 输出侧：imp，write() 立即处理
  uvm_analysis_imp #(compress_item, compress_scoreboard) output_imp;

  logic [127:0] exp_cache[string]; // 哈希缓存：key=原始输入十六进制字符串
  int cache_hit = 0;
  int cache_miss = 0;

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    input_export = new("input_export", this);
    input_fifo = new("input_fifo", this);
    output_imp = new("output_imp", this);
  endfunction

  function void connect_phase(uvm_phase phase);
    input_export.connect(input_fifo.analysis_export);
  endfunction

  // output 端：write() 从 input_fifo 取原始输入，查哈希缓存或计算期望，立即比对
  function void write(compress_item act);
    compress_item inp;
    logic [127:0] exp_out;
    string key;

    if (!input_fifo.try_get(inp)) begin
```

```
      `uvm_error("SB", "收到压缩输出但输入 fifo 为空！") return;
    end

    // 考点：哈希 key 由输入数据生成，相同输入直接复用缓存结果
    key = $sformatf("%0h", inp.raw_input);

    if (exp_cache.exists(key)) begin
      exp_out = exp_cache[key];
      cache_hit++;
    end else begin
      exp_out = ref_compress(inp.raw_input);
      exp_cache[key] = exp_out; // 写入缓存供后续复用
      cache_miss++;
    end

    if (act.compressed_out !== exp_out)
      `uvm_error("SB", $sformatf(
        "压缩结果不匹配: key=%s", key))
  endfunction

  // 参考压缩算法（XOR 折叠示例，实际项目替换为真实算法）
  function logic [127:0] ref_compress(logic [255:0] data);
    logic [127:0] out;
    for (int i = 0; i < 128; i++) out[i] = data[i] ^ data[i+128];
    return out;
  endfunction

  function void check_phase(uvm_phase phase);
    if (input_fifo.size() != 0)
      `uvm_error("SB", $sformatf(
        "输入 fifo 有 %0d 个事务未被消费！", input_fifo.size()))
  endfunction

  function void report_phase(uvm_phase phase);
    int total = cache_hit + cache_miss;
```

```
    `uvm_info("SB", $sformatf(
      "哈希缓存命中率: %.1f%%（命中 %0d 未命中 %0d 总计 %0d）",
      (total > 0) ? 100.0*cache_hit/total : 0.0,
      cache_hit, cache_miss, total), UVM_NONE)
  endfunction
endclass
```

【关键提示】

input（压缩引擎输入）→ analysis_fifo；output（压缩结果）→ imp。write() 中 try_get 取原始输入，查哈希缓存或调用参考模型，立即比对。比原来的 pkt_q+run_phase+semaphore 方案更简洁，缓存命中率统计仍完整保留。

题目 25 分层 Scoreboard 继承体系（基类与子类）

难度：高级

标签：pure virtual / virtual class / 三层继承 / factory override / 链式report

【题目描述】

在大型 SoC 验证项目中，不同子系统的 Scoreboard 往往共享相同的统计计数框架（总数、通过率、report 输出）和 AXI 协议合规检查逻辑（响应码校验、LAST 信号时序等），但各子系统的功能数据比对逻辑不同。需要设计一套三层继承的 Scoreboard 体系：第一层抽象基类仅提供统计框架，第二层中间类添加 AXI 协议合规检查，第三层叶子类实现具体 DUT 的功能数据比对。每层通过调用 super 触发父层逻辑，形成链式执行。本题考察 virtual class、pure virtual 函数、super 链式调用和 factory override 在分层 Scoreboard 架构中的综合运用。

【考察重点与实现要点】

> virtual class 不能被直接实例化，只有具体叶子类可以被 factory 创建
>
> pure virtual 在基类声明无函数体，编译器强制所有非抽象子类实现
>
> super.report_phase(phase) 链式调用确保每一层的统计都被打印
>
> factory override: dma_axi_scoreboard::type_id::set_type_override(other_sb::get_type())
>
> 第二层中 do_compare() 调用 do_functional_compare()，形成 "模板方法" 设计模式

【完整答案代码】

```
// ===== 第一层：通用统计框架（不可实例化）=====
virtual class sb_base_class extends uvm_scoreboard;
  int total_cnt = 0, pass_cnt = 0, fail_cnt = 0;

  // 考点：pure virtual，强制子类实现
  pure virtual function bit do_compare(
    uvm_sequence_item exp, uvm_sequence_item act);

  function void compare_and_count(
    uvm_sequence_item exp, uvm_sequence_item act);
    total_cnt++;
    if (do_compare(exp, act)) pass_cnt++;
    else fail_cnt++;
  endfunction

  // 考点：链式 report，子类调用 super 触发此函数
  function void report_phase(uvm_phase phase);
    `uvm_info("SB", $sformatf(
      "[基类统计] 总计:%0d 通过:%0d 失败:%0d",
      total_cnt, pass_cnt, fail_cnt), UVM_NONE)
  endfunction
endclass

// ===== 第二层：AXI 协议合规检查（模板方法模式）=====
class axi_sb_base extends sb_base_class;
  `uvm_component_utils(axi_sb_base)

  virtual function bit do_compare(
    uvm_sequence_item exp_obj, uvm_sequence_item act_obj);
    axi_item exp, act;
    $cast(exp, exp_obj); $cast(act, act_obj);
    // AXI 协议规则检查（所有子类共享）
    if (act.resp == SLVERR || act.resp == DECERR) begin
      `uvm_error("AXI", "收到错误响应码 SLVERR/DECERR")
      return 0;
```

```
    end
    if (act.valid && !act.last && act.beats_remaining == 0) begin
      `uvm_error("AXI", "LAST 信号时序错误")
      return 0;
    end
    return do_functional_compare(exp, act); // 调用子类功能比对
  endfunction

  pure virtual function bit do_functional_compare(
    axi_item exp, axi_item act);

  function void report_phase(uvm_phase phase);
    super.report_phase(phase); // 考点：链式调用，触发第一层统计
    `uvm_info("SB", "[AXI层] 协议检查完成", UVM_NONE)
  endfunction
endclass

// ===== 第三层：具体 DUT 功能比对（叶子类）=====
class dma_axi_scoreboard extends axi_sb_base;
  `uvm_component_utils(dma_axi_scoreboard)

  uvm_analysis_imp #(axi_item, dma_axi_scoreboard) analysis_export;
  axi_item exp_q[$];

  function void build_phase(uvm_phase phase);
    super.build_phase(phase);
    analysis_export = new("analysis_export", this);
  endfunction

  function void write(axi_item actual);
    axi_item expected;
    if (exp_q.size() == 0) begin
      `uvm_error("SB", "无期望值") return;
    end
    expected = exp_q.pop_front();
```

```
    compare_and_count(expected, actual); // 调用基类框架
  endfunction

  // 实现具体功能比对
  function bit do_functional_compare(axi_item exp, axi_item act);
    if (act.data !== exp.data || act.strb !== exp.strb) begin
      `uvm_error("SB", $sformatf(
        "DMA 数据错误: exp_data=0x%0h act_data=0x%0h",
        exp.data, act.data))
      return 0;
    end
    return 1;
  endfunction

  function void report_phase(uvm_phase phase);
    super.report_phase(phase); // 触发第二层和第一层的统计
    `uvm_info("SB", "[DMA 功能层] 验证完成", UVM_NONE)
  endfunction
endclass

// ===== Factory Override（运行时替换实现，无需改代码）=====
// dma_axi_scoreboard::type_id::set_type_override(alt_sb::get_type());
```

【关键提示】

三层继承遵循"模板方法"设计模式：基类定义算法骨架（compare_and_count），中间层添加协议检查，叶子类提供具体功能比对。每层 report_phase 调用 super.report_phase() 形成链式打印，确保所有层的统计都被输出。virtual class 防止基类被误实例化，pure virtual 保证叶子类必须实现比对逻辑。